PACE UNIVERSITY

SEIDENBERG SCHOOL OF CSIS

Regularization & Stability

Dr. Krishna Bathula

Learning Objectives

1. Learning Curves
2. R-Square
3. Feature Selection
4. **R**egularized **L**oss **M**inimization
5. Ridge Regression
6. Lasso Regression
7. Elastic Net
8. Stability

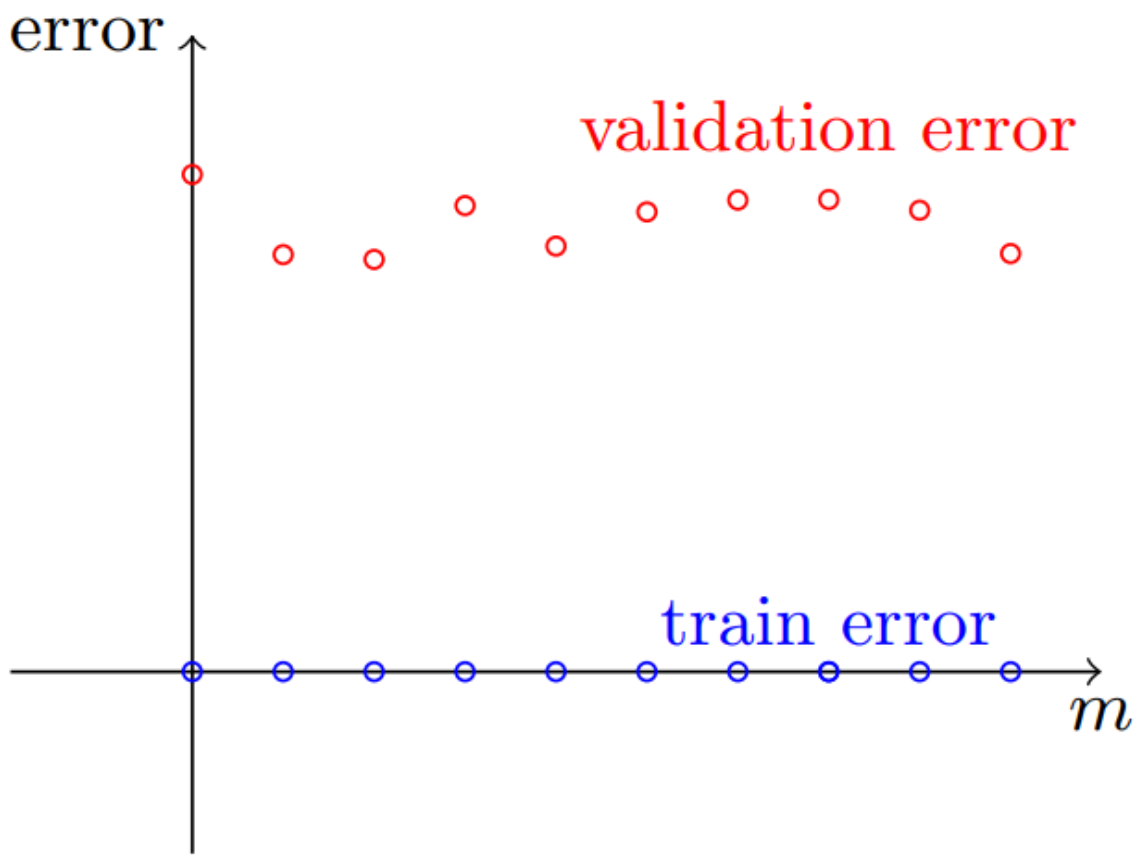
Model Performs Bad?

What to do, if Learning still fails even after considering the following scenarios:

- Given a learning task we have approached it with a choice of a **hypothesis** class, a **learning algorithm**, and **parameters**.
- A **validation set** is used to tune the parameters and then test the learned predictor on a test set. (includes managing model complexity)
- The test **results**, unfortunately, turn out to be unsatisfactory.
- What went wrong then, and what should we do next?

Learning Curves

A Learning Curve is plot between the data samples m and the training error of the Learner (Algorithm).



Learning Curve with 10 Samples

Learning Curves (Model Selection Curve)

To produce a Learning Curve, we train the algorithm on prefixes of the data of **increasing sizes**.

For example, we can first train the algorithm on the first **10% of the examples**, then **on 20%** of them, and so on.

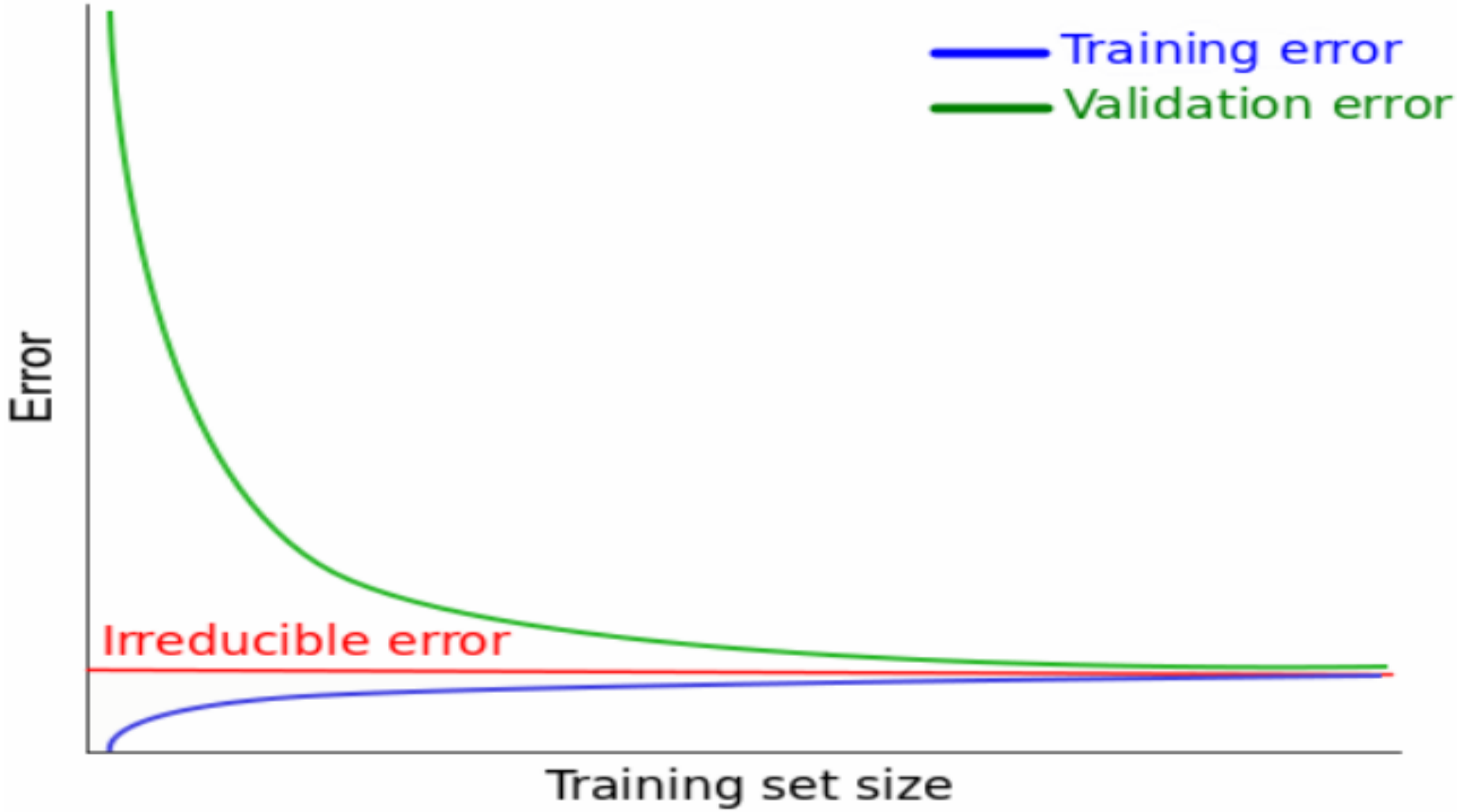
For each prefix we **calculate the training error** (on the prefix the algorithm is being trained on) and the **validation error** (on a predefined validation set).

Such learning curves can help us distinguish between the two scenarios of having fewer samples and more samples.

In the **fewer sample situation**, we expect the validation error to be approximately 1/2 for all prefixes, as the algorithm **didn't learn anything**.

In the second scenario the validation error will **start as a constant** but then should **start decreasing**.

Learning Curve



Model Still not Generalized?

Many elements can be “fixed.” The main approaches are:

1. Get a larger sample

2. Change the hypothesis class by: –
   - Enlarging it
   - Reducing it
   - Completely changing it
   - Changing the parameters considered

3. Change the **optimization algorithm** used to apply the learning rule

4. Change the **feature representation** of the data

5. **Hyperparameters**

2.1 & 4 Increases Model Complexity

Checkpoints

1. If learning involves **parameter tuning**, plot the model-selection curve to make sure that you tuned the parameters appropriately.

2. If the **training error is excessively large** consider **enlarging the hypothesis** class, completely changing it, or changing the **feature representation** of the data.

3. If the **training error is small**, **plot learning curves** and try to deduce from them whether the problem is with an estimation error or approximation error.

4. If the **approximation error** seems to be **small** enough, try to **obtain more data**. If this is not possible, consider reducing the complexity of the hypothesis class.

5. If the **approximation error** seems to be **large** as well, try to change the **hypothesis class** or the **feature representation** of the data completely.

Model Evaluation

- How accurate the Model is with respect to **new data**?
- Consider the Linear Regression Models.
- Are we able to minimize the **empirical loss** and **true loss** by the methods discussed so far?
- Are we using the **right features** in predicting accurate outputs?
- Can we evaluate the model to check if it is generalized and accurate?
- An evaluation Metric to check the feature selection and Model complexity is **R-Square**.

Model Evaluation Metrics – Linear Regression

- The analysis method estimates the relationship between **independent variables** (Features) and a **dependent variable** (Target) termed **Ordinary least squares** (**OLS**) regression.

- The process predicts the ties by minimizing the **sum of the squares** in the difference between the **observed** and **predicted** values of the dependent variable.

- On the other hand, the linear regression model, whose coefficients are not estimated by OLS but by an estimator, is biased.

Model Evaluation Metrics – Linear Regression

- In regression modeling, the presence of multicollinearity significantly leads to inconsistent parameter estimates.
- Ordinary Least Squares (OLS), a standard method in regression analysis, results in an inaccurate and unstable model because it is not robust to the multicollinearity problem.
- Several methods have been proposed in the literature to address this model instability issue, and the most common one is ridge regression.

Interpretation of Linear Regression Evaluation Metrics

- The **Mean Absolute Error** (**MAE**) measures the average absolute magnitude between the actual values and the predicted values by regression model.

- The MAE can be written mathematically as

$$\mathbf{MAE = (1/n) * \Sigma |y_i - h(x_i)|}$$

- where:

  - n = total numbers of observation
  - y_i = actual value for the i^{th} observation
  - $h(x_i)$ = predicted value for the i^{th} observation

Interpretation of Evaluation Metrics - MAE

- The MAE is 5 meaning that there's 5 values difference between the actual and predicted values on average.
- The **lower the MAE**, the **better** the **model predicts**.
- However, the relationship between MAE values and how well a model performs **depends on the data**.
- For example, an **MAE value of 500** is very good for prediction of the house prices considering the average house prices range in **hundreds of thousands** of dollars.
- However, an **MAE value of 500 will be a poor** prediction if it's for a stock price prediction that **ranges around $700 to $1000**.

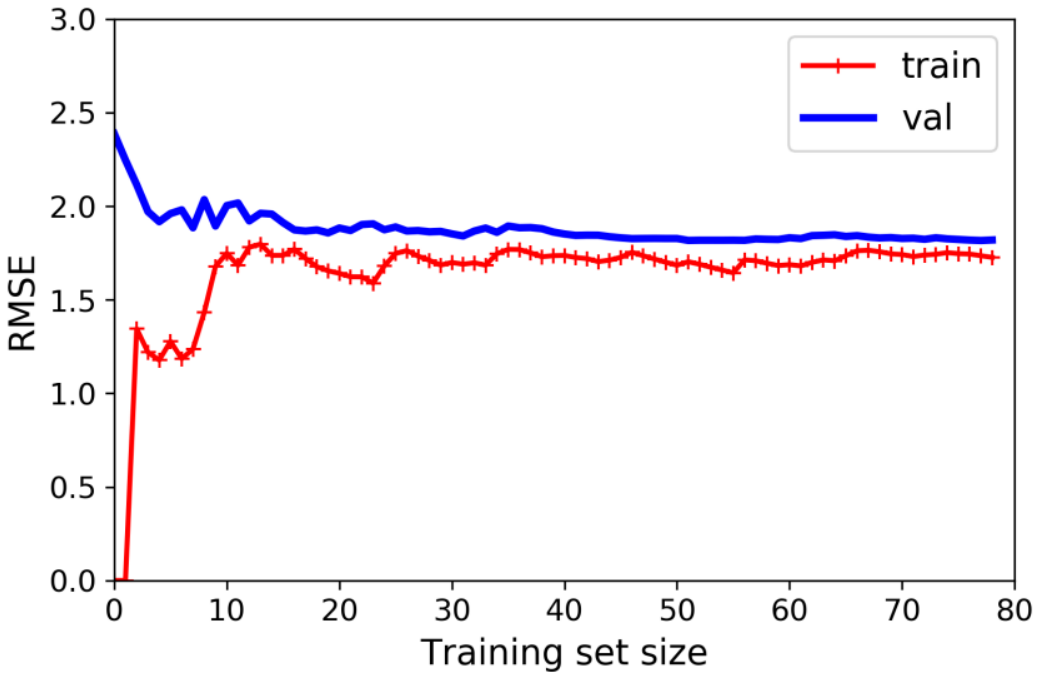
Interpretation of Evaluation Metrics – MSE & RMSE

- The **Mean Squared Error** (**MSE**) is the average of the squared difference between the actual value and predicted values by the regression model.
- The reason for the squaring of the error is to remove any negative signs.
- By squaring the error, **MSE** penalizes the error more than **MAE**.
- **RMSE** is the error value obtained by the square root of **MSE**. **RMSE** is the most widely used metric in the regression analysis.
- The reason for using the **root of the error** is to have the error in the same unit as the outcome variable for easier interpretation purposes.

Interpretation of Evaluation Metrics - MSE & RMSE

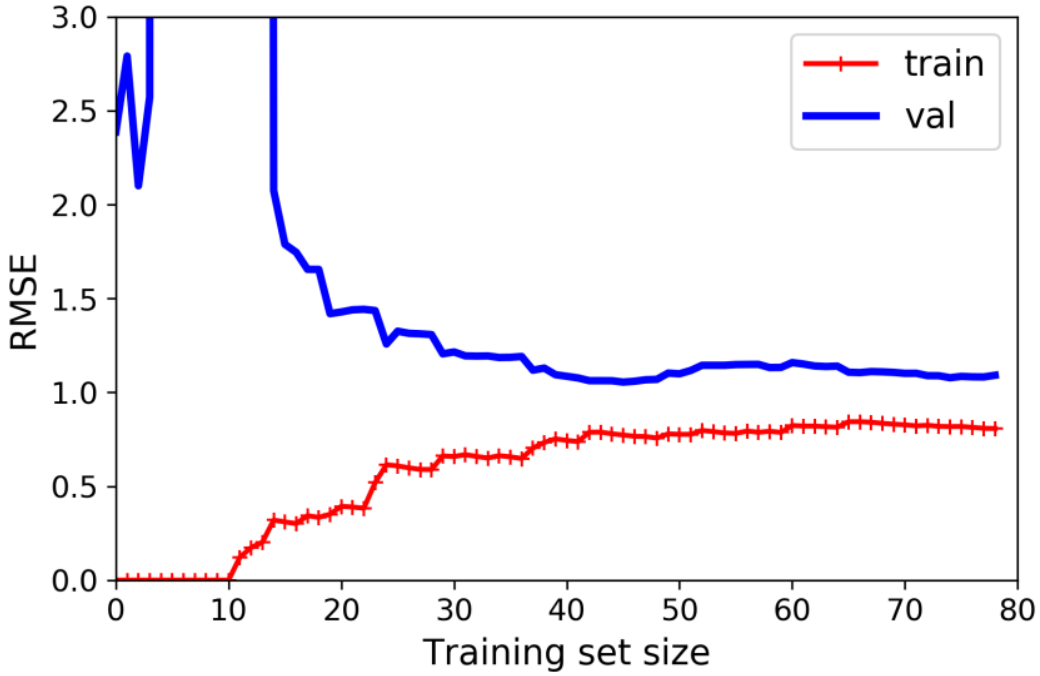
- Both RMSE and MSE are useful to see if the **outliers** are interfering with the model's predictions.
- The metrics inform how close the **predicted values** are to the **regression line**.
- The **closer the point** is to the regression, the **lower the metrics values** are and the higher the accuracy of the regression model is.
- When a model is **100% perfect**, then the **metrics values** will be equal to **zero**.
- Thus, as the same concept as MAE, the **lower the MSE or RMSE**, the **better the model predicts**.

Learning Curve - Regression



Example: Polynomial regression of degree 1

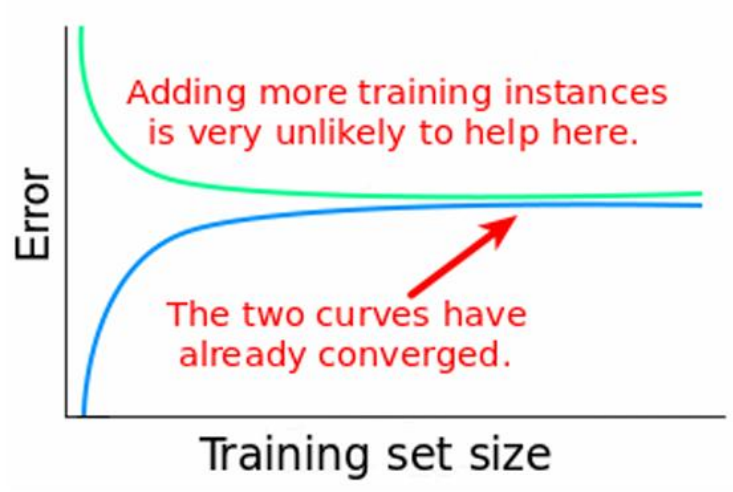
Underfitting

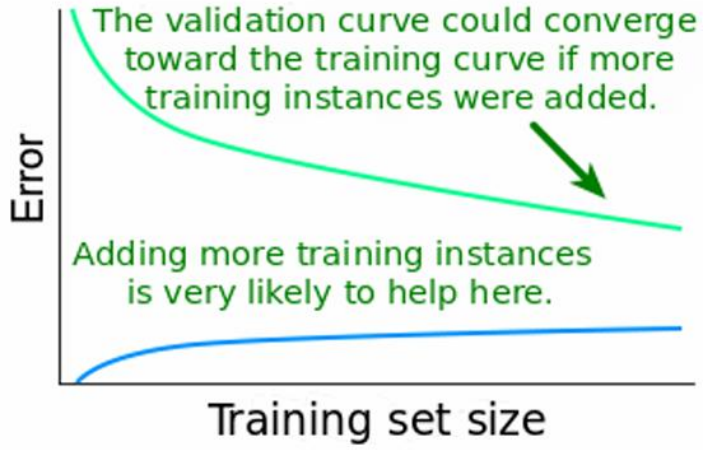


Example: Polynomial regression of degree 10

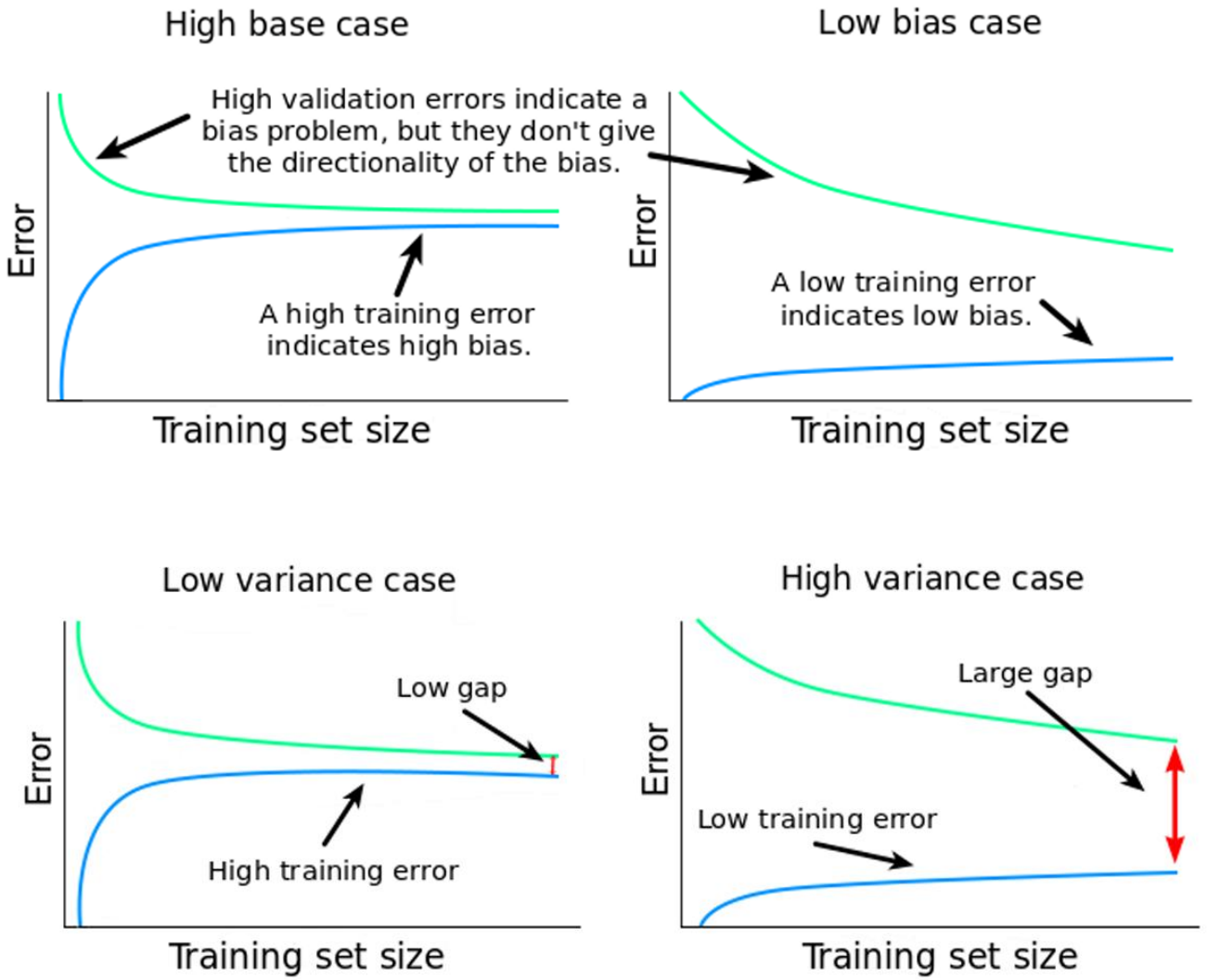
Overfitting

Learning Curve – Linear Regression Model





Learning Curves for Bias-Variance



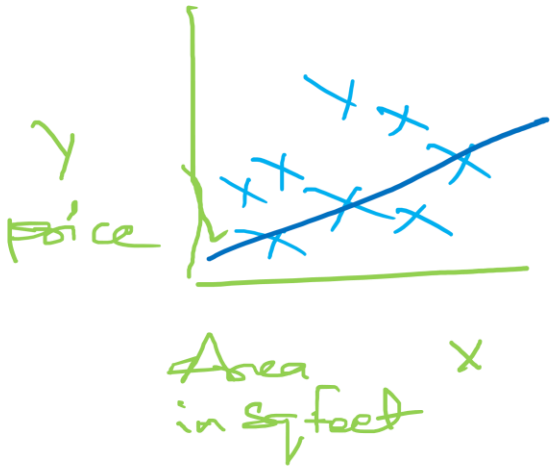
R-Square

- **R-Square** determines how much of the total variation in the dependent variable **Y** is explained by the variation in the independent variable **X**.
- Example Housing price and area of the house in square feet.
- Is area the only feature to impact the house price? What is the **coefficient estimate** (impact index) of this feature?
- The mathematical representation of R-Square is :

$$R - Square = 1 - \frac{\sum(Y_{actual} - Y_{predicted})\text{^}2}{\sum(Y_{actual} - Y_{mean})\text{^}2}$$

Example – Adding More Features

Linear Regression Example: Predicting House prices, with the Area of the house in square feet. Additional Features selection → No. of Bedrooms, Age of the house,



Area = 68% ↑

+ New feature

No. of Bedrooms

Accuracy + 12%

Total = 68 + 12 = 80%

+ Age

New Build +

Woodwork +50,000$

old ↓ depreciation -30,000$

Accuracy = 14% +

Total = 94%

4 more feature

* Total Built-up Area
* Lawn / Backyard
* Swimming Pool
* Fountain

} 5% × 4 = 20%

94% + 20% = 114%

<100
>1000
>10,000

R-Square

- The **value of R-square** is always between **0 and 1**, where **0** means that the model does not explain **any variability** in the target variable (Y), and **1** means it explains **full variability** in the target variable.
- What happens if we just keep adding the features? Is it a good strategy?
- There is an increase in the value R-square, does it mean that the addition of **more features is useful for our model**?
- Having a checkpoint will help in understanding if the number of new features added will increase the accuracy.
- This introduces the approach of **Adjusted R-Square**.

Adjusted R-Square

- The only drawback of R-Square is that if **new features are added** to our model, R-Square only **increases** or remains constant but it never decreases.
- We can not judge that by **increasing the complexity of our model**, are we making it **more accurate**?
- So, a more reliable approach is Adjusted R-Square.
- The Adjusted R-Square is the modified form of R-Square that has been adjusted for the number of features in the model.
- It incorporates the **model's degree of freedom**.

Adjusted R-Square Expression

The adjusted R-Square only increases if the new term improves the model accuracy.

$$R^2 \text{ adjusted} = 1 - \frac{(1 - R^2)(N - 1)}{N - p - 1}$$

Where,

- R2 = Sample R square
- p = Number of features
- N = total sample size

Selecting Right Features

- When we have a **high dimensional data set**, it would be **highly inefficient** to use **all the variables** since some of them might be imparting **redundant information**.
- We would need to select the **right set of independent** variables that give us an **accurate model** and are able to explain the dependent variable well.
- There are multiple ways to select the right set of variables for the model.
- The variables we're selecting **should not be correlated** among themselves.
- Instead of manually selecting the variables, we can automate this process by using forward or backward selection.

Summary

With only the **necessary** **Feature selection**, there will be a great improvement in both **MSE** and **R-square**, which means that the model is able to **predict** much **closer values** to the **actual values**.

Model selection refers to the process of **choosing the model that best generalizes**. The model generalizes well with cross-validation techniques.

Overfitting happens when our model **performs well on our training dataset but generalizes poorly**.

Underfitting happens when our model **performs poorly on both our training dataset and unseen data**.

Parameters

Parameters are **internal** to the model.

That is, they are **learned** or **estimated** purely from the data during training '$\boldsymbol{\theta}$'.

The ML algorithm used tries to learn the mapping between the **input features** and the **labels** or **targets**.

Model training typically starts with **parameters** being initialized to some values (random values or set to zeros).

As training/learning progresses the **initial values are updated** using an **optimization algorithm** (e.g. **gradient descent**).

Examples - Parameters

The learning algorithm is continuously updating the parameter values as learning progress

Examples:

- The **coefficients** (or **weights**) of linear and logistic regression models.
- **Weights** in a Neural Network(NN)
- The **cluster centroids** in clustering

Parameters

Parameters in machine learning and deep learning are the values your learning algorithm can change independently as it learns

These values are **affected by the** choice of hyperparameters you provide.

Parameters are continuously **being updated**, and the final ones at the end of the training constitute your model.

Gradient Descent **tweaks parameters iteratively** to minimize a cost function.

It measures the local gradient of the error function with respect to the parameter vector θ, and it goes in the direction of the descending gradient.

Once the **gradient is zero**, you have reached a **minimum**.

Hyperparameters

Hyperparameters are parameters whose values control the learning process and determine how well the learning algorithm learns the values of model **parameters**.

The prefix '**hyper_**' refers to 'top-level' parameters that control the learning process and the model **parameters** that result from it.

The **hyperparameters** are set **before the model's training** and are used by the learning algorithm.

They are said to be **external to the model** as the model **cannot change** its values during learning/training.

Hyperparameters are those we configure before training begins, and these values or **configurations will remain the same** when training ends.

Examples - Hyperparameters

- Train-test split ratio
- Learning rate in optimization algorithms (e.g. gradient descent)
- Choice of optimization algorithm (e.g., gradient descent, stochastic gradient descent)
- The choice of cost or loss function the model will use
- Number of clusters in a clustering task

Neural Networks

- Choice of activation function in a neural network (nn) layer (e.g. Sigmoid, ReLU, Tanh)
- Number of hidden layers in a NN
- Number of activation units in each layer
- The drop-out rate in nn (dropout probability)
- Kernel or filter size in convolutional layers
- Number of iterations (epochs) in training a NN
- Pooling size
- Batch size

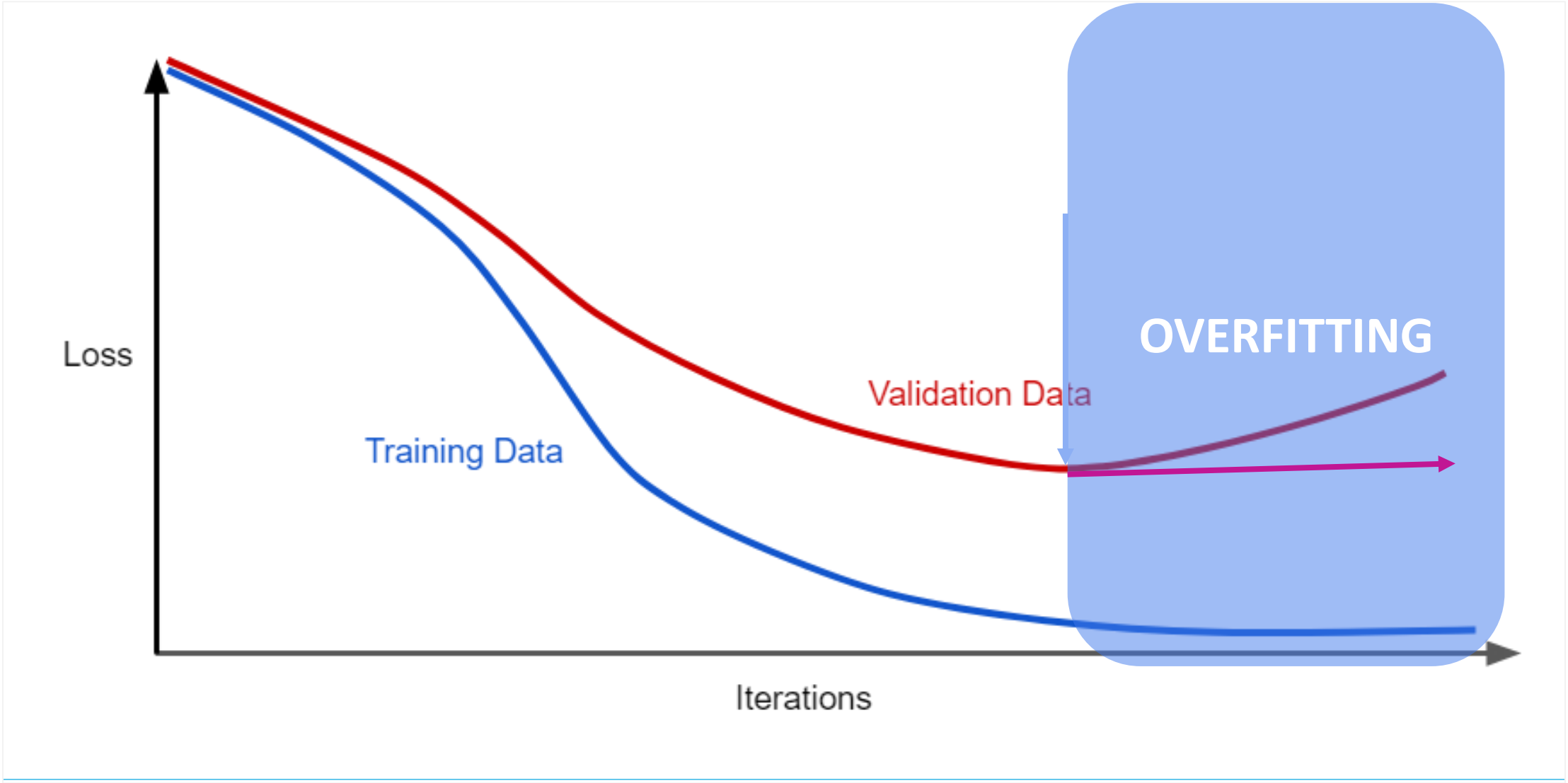
Hyperparameters

Hyperparameters have an **impact on the parameters** your model learns.

Setting the **right hyperparameter values** is important because it directly **impacts the model's performance**

The process of choosing the best hyperparameters for your model is called **hyperparameter tuning**

Generalization Curve



Overfitting - Prevention

How can we prevent the Model from overfitting?

The training optimization algorithm is a function of two terms:

1. The **Loss** term, which measures how well the model fits the data, and
2. The **Regularization** term, which measures model complexity

Optimization Algorithm = Minimize(Loss + Complexity)

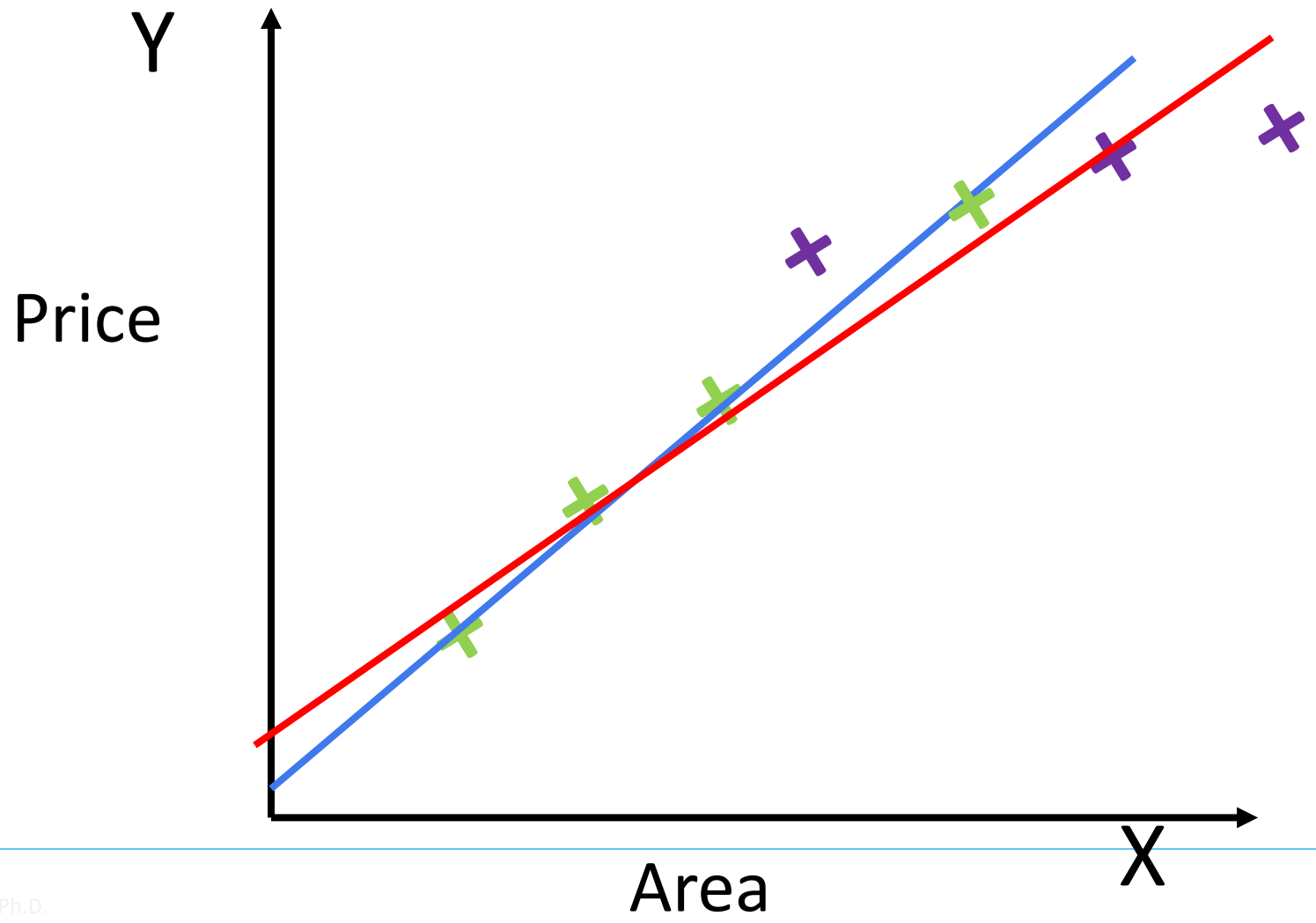
We could prevent overfitting by penalizing complex models, a principle called **Regularization**

Regularization

- A good way to **reduce overfitting** is to **Regularize** the model.
- For example, a simple way to regularize a polynomial model is to reduce the number of polynomial degrees.
- The fewer **degrees of freedom** it has, the harder it will be for it to overfit the data.
- For a linear model, regularization is typically achieved by **constraining the weights of the model**.
- Constraining a model to make it simpler and reduce the risk of overfitting is called **Regularization**

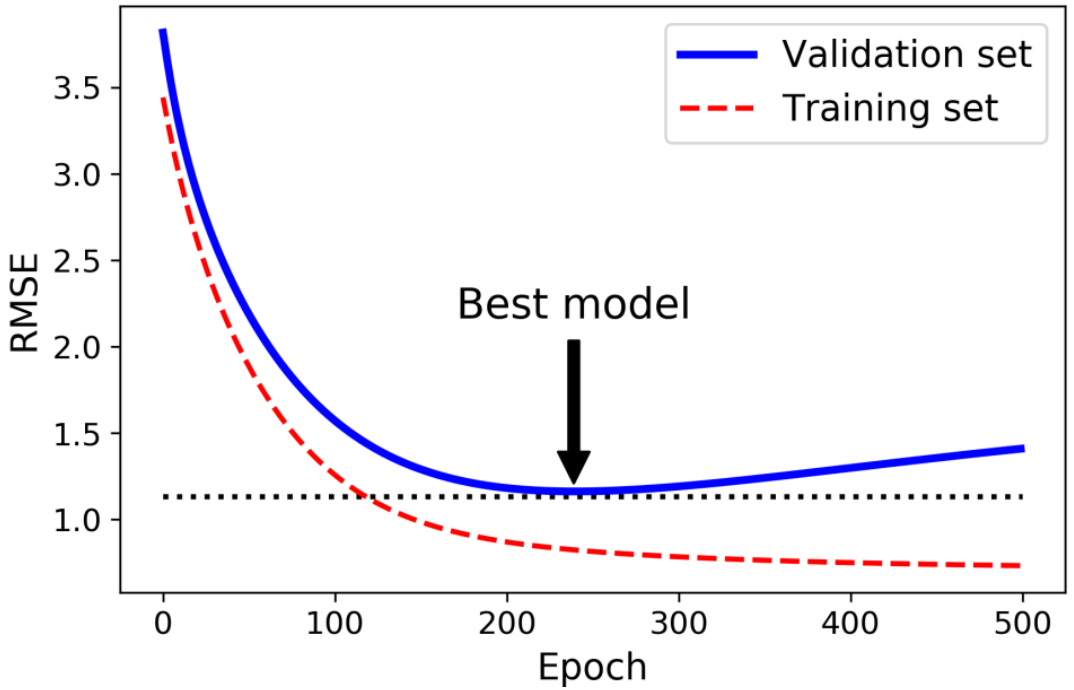
Regularization vs Overfitting

How can Regularization control overfitting?



Early Stopping

One way to **regularize iterative learning algorithms** such as Gradient Descent is to **stop training** as soon as the validation error reaches a minimum.
This is called **Early Stopping**.



With early stopping, we just stop training as soon as the validation error reaches the minimum.

Regularized Loss Minimization (RLM)

- **Regularized Loss Minimization** (RLM) is a learning rule in which we jointly minimize the empirical risk and a regularization function.

- Formally, a regularization function is a mapping **R: R_d → R**, and the regularized loss minimization rule outputs a hypothesis in

$$\operatorname*{argmin}_{\mathbf{w}} \left(L_S(\mathbf{w}) + R(\mathbf{w}) \right)$$

- The '**complexity**' of hypotheses is measured by the value of the **regularization function**, and the algorithm balances between low empirical risk and '**simpler**' or '**less complex**' hypotheses.

- Our training **optimization algorithm** is now a function of two terms:
  - The **Loss** term, which measures how well the model fits the data, and
  - The **Regularization** term, which measures model complexity.

Regularization Function

One of the simplest regularization functions is given by

$$\boldsymbol{R(\theta) = \alpha||\theta||^2}$$

$$R(\mathbf{w}) = \lambda\|\mathbf{w}\|^2$$

Where λ > 0 is a scalar $\boldsymbol{||\theta|| = \sqrt{\sum_{i=1}^{n} \theta_i^2}}$

$$\|\mathbf{w}\| = \sqrt{\sum_{i=1}^{d} w_i^2}$$

This yields the learning rule: $$A(S) = \underset{\mathbf{w}}{\operatorname{argmin}} \left(L_S(\mathbf{w}) + \lambda\|\mathbf{w}\|^2 \right)$$

Where the norm of $\boldsymbol{\theta}$ is a measure of its "complexity."

Ridge Regression

- Linear regression may encounter problems in which the model's estimated coefficients can become **relatively large**, making the model so unstable that it becomes **sensitive** to the inputs.
- An approach can be adopted to regain the regression **model's stability**.
- In which the **loss function** is modified and includes **additional costs** for a model with relatively **large coefficients**.
- This type of regularization function is often called **Tikhonov** regularization or **Ridge Regularization**.

Ridge Regression

- **Small coefficient** values, **penalties** added to the loss function during the training period.
- These extensions were termed as penalized linear regression or **regularized linear regression**.
- **Ridge regression** uses the **L2** penalty, which shrinks the coefficients of input variables that have **not contributed to the prediction task**.
- This forces the learning algorithm to **fit the data** and keep the model weights as small as possible.

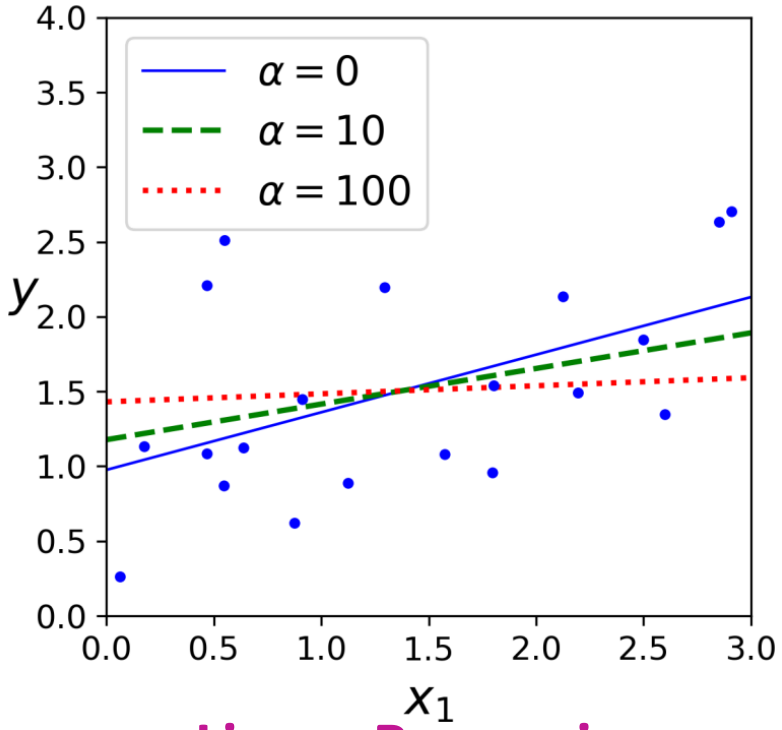
Ridge Regression

- Ridge Regression (also called Tikhonov regularization) is a regularized version of Linear Regression:
- A regularization term equal to $\alpha \sum_{i=1}^{n} \theta_i^2$ is added to the **cost function**.
- This forces the learning algorithm to not only **fit the data** but also keep the model **weights as small as possible**.
- The regularization term should **only be added** to the cost function **during training**.
- Once the model is trained, you want to evaluate the model's performance using the **unregularized performance measure**.

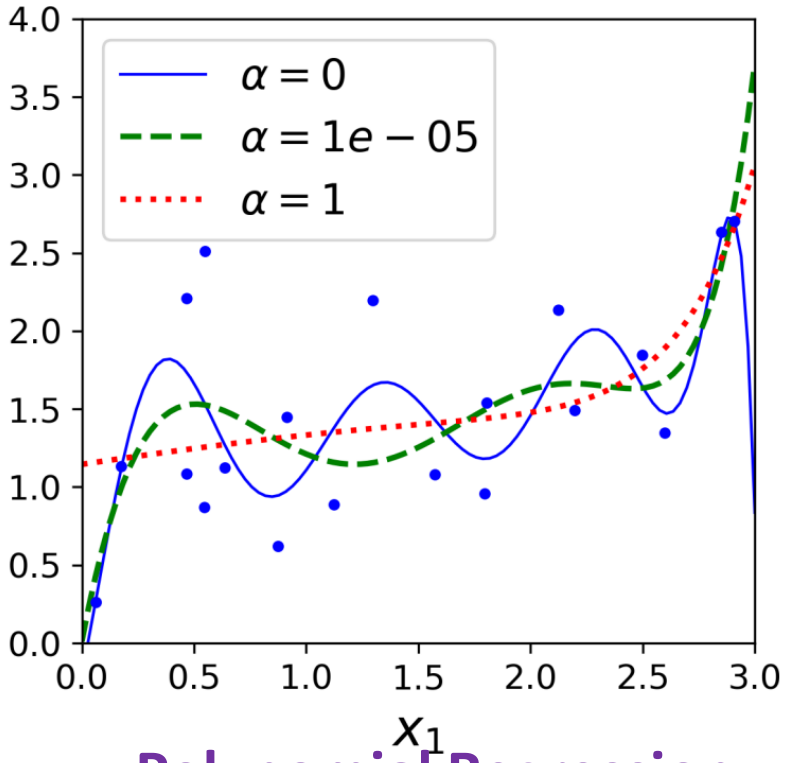
Ridge Regression

The hyperparameter '**α**' controls how much you want to regularize the model.

- If **α = 0**, then Ridge Regression is just **Linear Regression**.
- If **α** is very large, then all weights end up very close to zero and the result is a flat line going through the data's mean.



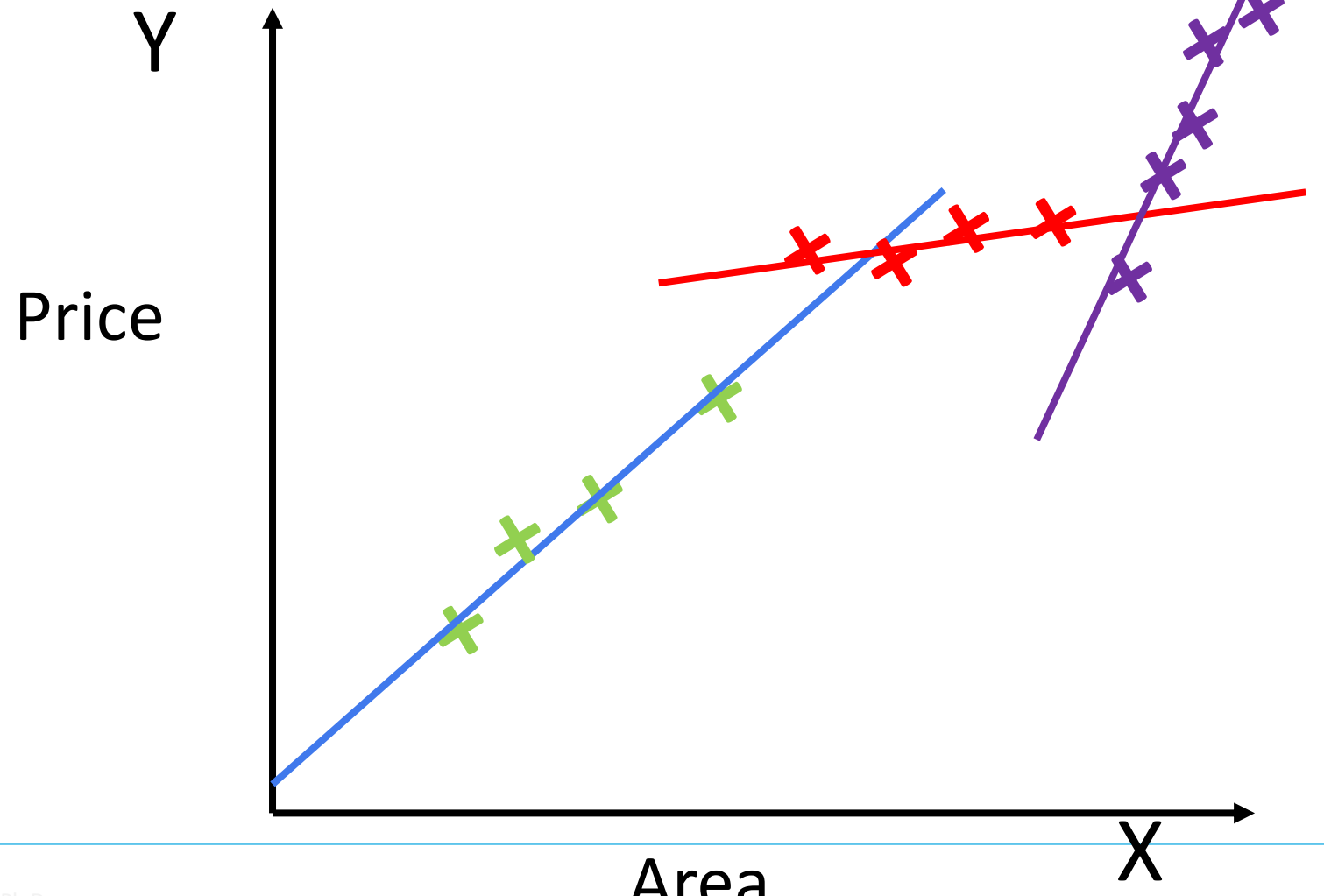
Linear Regression



Polynomial Regression

Ridge Regression – Controlling Model Overfitting

How can Regularization control overfitting?



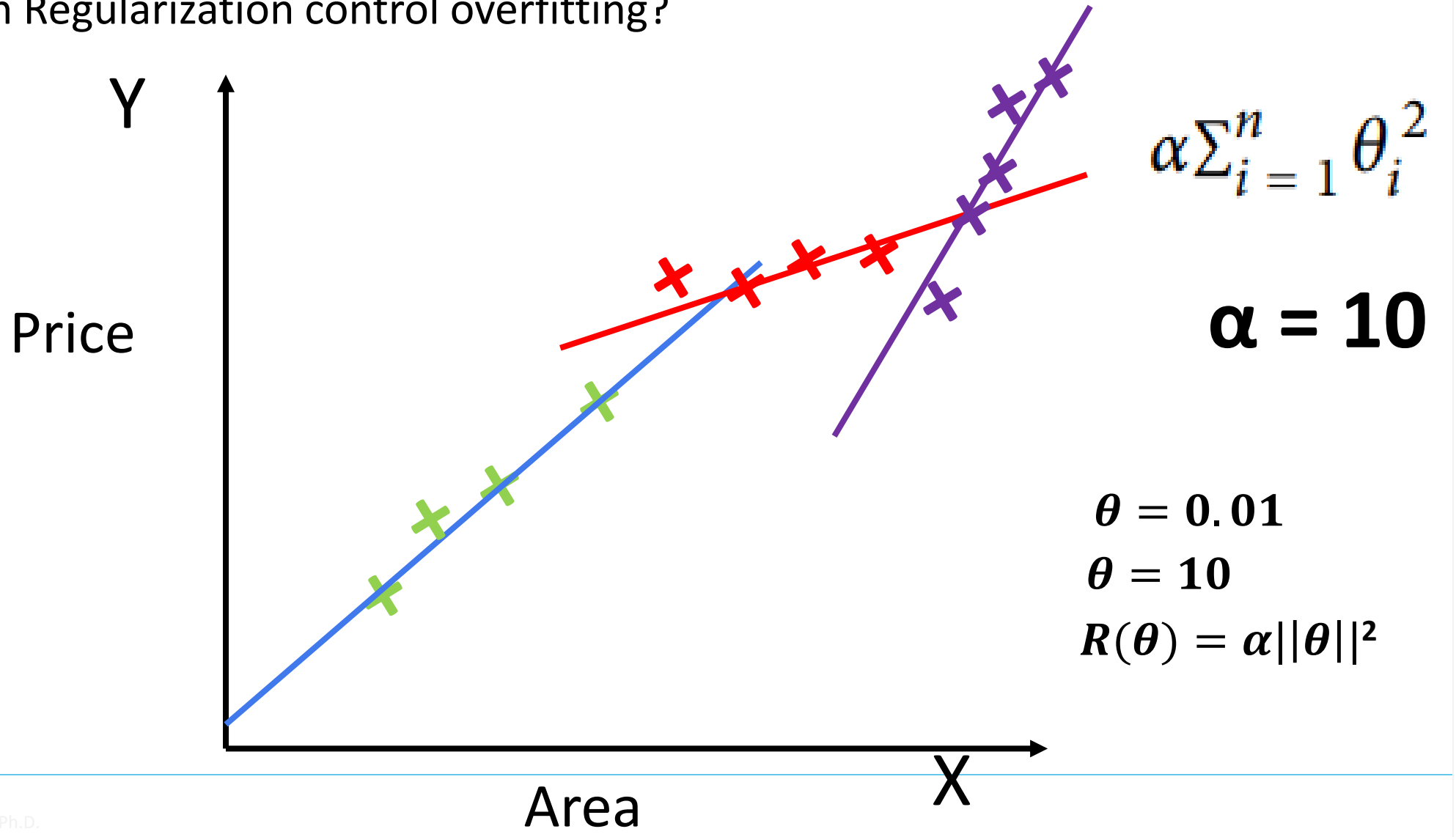
$$R(\theta) = \alpha||\theta||^2$$

$$\alpha \Sigma_{i=1}^{n} \theta_i^2$$

α = 0

Ridge Regression – Controlling Model Overfitting

How can Regularization control overfitting?



$$\alpha \Sigma_{i=1}^{n} \theta_i^2$$

α = 10

$$\boldsymbol{\theta = 0.01}$$

$$\boldsymbol{\theta = 10}$$

$$\boldsymbol{R(\theta) = \alpha ||\theta||^2}$$

Lasso Regression

- **L**east **A**bsolute **S**hrinkage and **S**election **O**perator Regression (Lasso Regression) is another regularized version of Linear Regression.

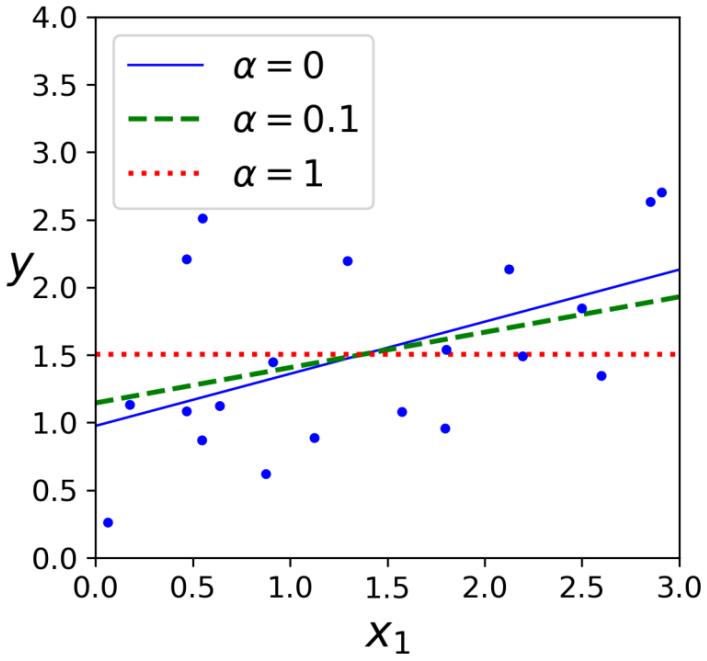
- Unlike Ridge Regression, it adds a regularization term to the cost function, but it uses the ℓ1 norm of the weight vector instead of half the square of the ℓ2 norm

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \alpha \Sigma_{i=1}^{n} \left|\theta_i\right|$$

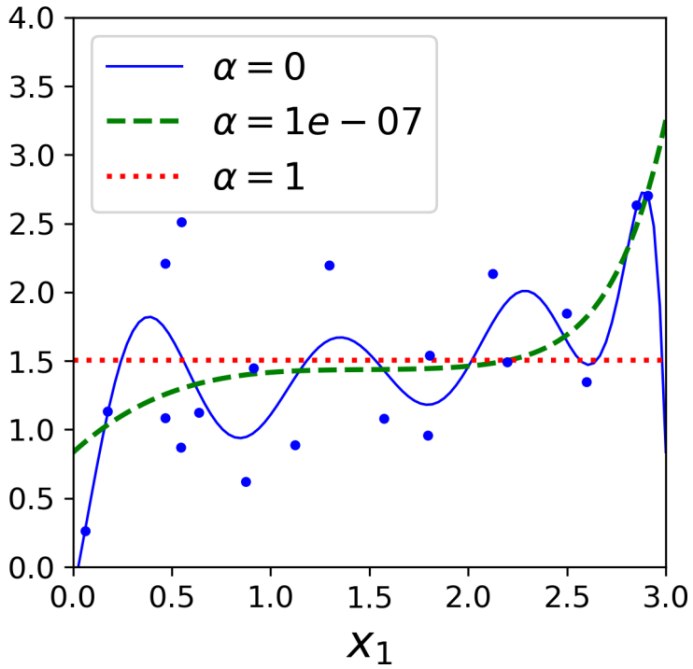
- An important characteristic of Lasso Regression is that it tends to eliminate the weights of the least important features

Lasso Regression

- The dashed line in the right plot (with α = 1e-07) looks quadratic, almost linear.
- All the weights for the high-degree polynomial features are equal to zero.
- In other words, Lasso Regression automatically performs feature selection and outputs a sparse model.



Linear Regression



Polynomial Regression

Elastic Net

Elastic Net is a middle ground between **Ridge Regression** and **Lasso Regression**.

The regularization term is a simple **mix of both Ridge and Lasso's regularization** terms, and you can control the mix ratio **r**.

When **r = 0**, Elastic Net is equivalent to Ridge Regression, and when **r = 1**, it is equivalent to Lasso Regression

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + r\alpha\Sigma_{i=1}^{n}\left|\theta_i\right| + \frac{1-r}{2}\alpha\Sigma_{i=1}^{n}\theta_i^2$$

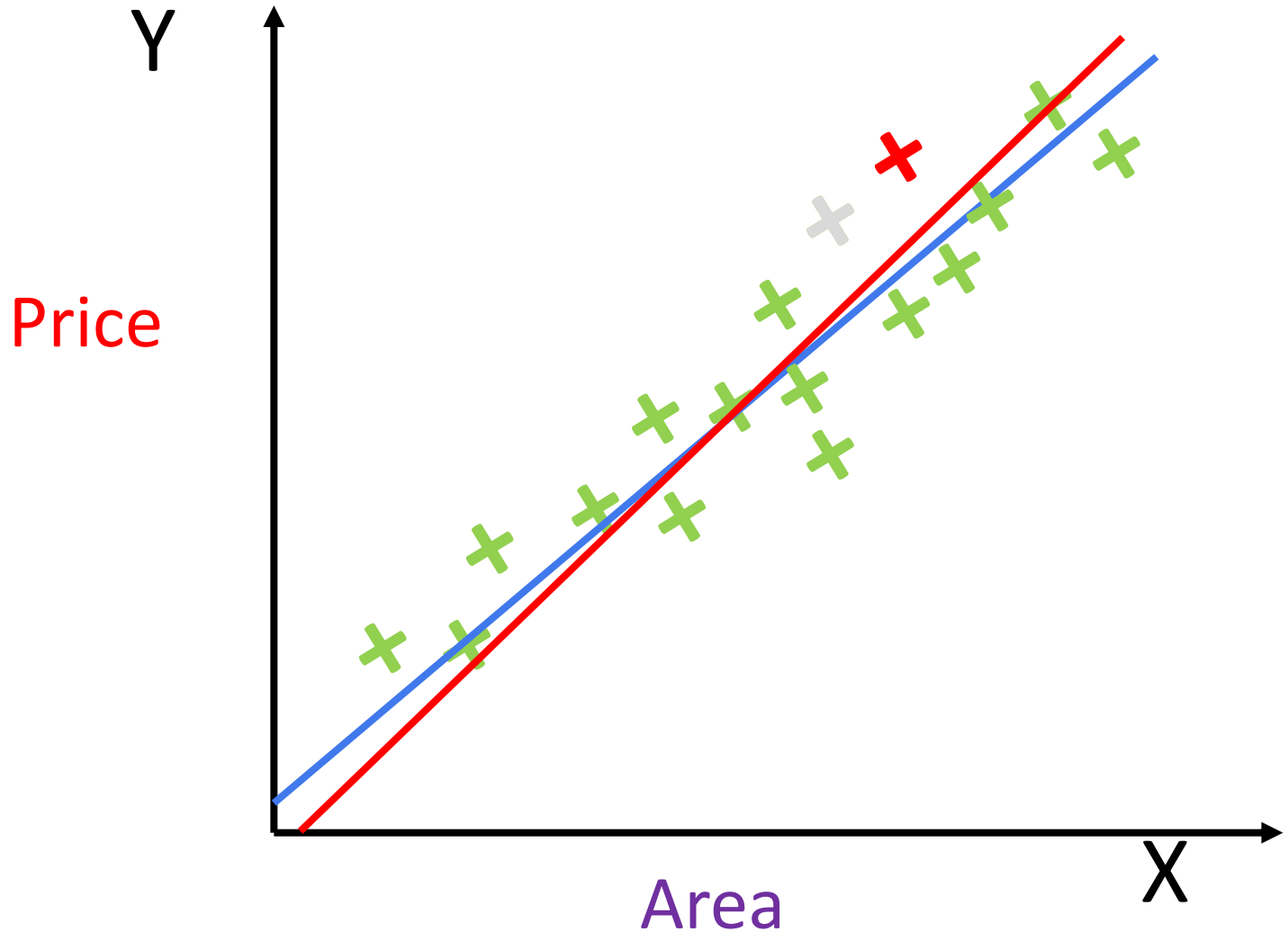
Stable Rules Do Not Overfit

- A learning algorithm is stable if a small change of the input to the algorithm does not change the output of the algorithm much.
- Let A be a learning algorithm, let $\mathbf{S = (z_1, \ldots, z_m)}$ be a training set of m examples, and let **A(S)** denote the output of **A**.
- Algorithm A suffers from overfitting if the difference between the true risk of its output, $L_D(A(S))$, and the empirical risk of its output, $L_S(A(S))$, is large.
- Given the training set S and an additional example z', let S (i) be the training set obtained by replacing the i'th example of S with z' ; namely, $\mathbf{S^{(i)} = (z_1, \ldots, z_{i-1}, z', z_{i+1}, \ldots, z_m)}$.
- "A small change of the input" means that we feed **A** with $\mathbf{S^{(i)}}$ instead of with **S**

Stable Rules Do Not Overfit

- That is we only replace one training example.
- We measure the effect of this small change of the input on the output of A, by comparing the loss of hypothesis **A(S)** on $\mathbf{z_i}$ to the loss of hypothesis $\mathbf{A(S^{(i)})}$ on $\mathbf{z_i}$.
- A good learning algorithm will have $\boldsymbol{l(A(S^{(i)}), z_i) - l(A(S), z_i) \geq 0}$, since in the first term the learning algorithm does not observe the example $\mathbf{z_i}$ while in the second term, $\mathbf{z_i}$ is indeed observed.
- If the preceding difference is very large, we suspect that the learning algorithm might overfit
- The reason for the large difference is that the learning algorithm drastically changes its prediction on $\mathbf{z_i}$ if it observes it in the training set.

Change of Input vs Stability – Example



Stability

- A learning algorithm that **does not overfit** is **not** necessarily a **good learning algorithm**.
- Ex: Algorithm **A** always outputs the same hypothesis.
- A useful algorithm should find a hypothesis that on the one hand fits the training set with **low Empirical risk** and on the other hand **does not overfit**.
- The algorithm should both **fit the training set** and at the same time be **stable**.
- Applying the parameter of the **RLM** rule balances between fitting the training set and being stable.

END